

第11讲 项目架构与移动平台工程实践

一、课程基本信息

项目	内容
课程名称	移动应用开发
授课章节	第六章 综合开发实践（第1讲）
授课学时	2学时
授课方式	理论讲授 + 案例分析
授课教师	刘东良
适用专业	软件工程

二、教学目标

知识目标

- 掌握MVP/MVVM架构模式的核心思想与在移动应用中的实践方法
- 理解移动应用分层架构设计原则与各层职责划分
- 掌握主流移动平台的数据存储方案及其适用场景
- 理解移动应用权限管理、生命周期管理与后台任务处理机制
- 了解推送通知、蓝牙与WiFi通信的基本原理与集成方式

能力目标

- 能够根据项目需求选择合适的架构模式并进行分层设计
- 能够针对不同数据类型选择恰当的存储方案并实现
- 能够实现运行时权限请求与应用生命周期状态管理
- 能够集成推送通知与基础的蓝牙/WiFi通信功能
- 具备移动平台工程实践的问题分析与解决能力

素质目标

1. 培养系统化的架构思维与工程设计能力
2. 树立严谨的质量意识与规范开发习惯
3. 增强对多平台技术差异的包容与适应能力

三、教学重点与难点

教学重点

1. MVVM架构模式的核心思想与实践方法
2. 移动应用分层架构设计原则
3. 各类数据存储方案的特点与适用场景
4. 运行时权限管理与应用生命周期管理
5. 后台任务处理机制

教学难点

1. MVP与MVVM模式的本质区别与选型决策
2. 数据层与业务层的解耦设计
3. 不同平台数据存储方案的差异与统一
4. 后台任务的生命周期与系统限制
5. 蓝牙通信的状态管理与异常处理

四、教学内容与过程设计

第一学时：项目架构设计与数据存储方案（45分钟）

1. 课程导入（5分钟）

- **问题导入**：随着项目功能越来越复杂，代码变得臃肿难维护，该怎么办？
- **案例分析**：一个"面条式代码"的反例项目 vs 一个架构清晰的项目对比
- **学习目标**：掌握架构设计方法，学会打造可维护、可扩展的移动应用
- **知识地图**：架构模式 → 分层设计 → 数据存储 → 平台工程实践

2. 项目架构设计 (20分钟)

为什么需要架构设计？

- 代码可维护性：降低理解成本，便于修改扩展
- 团队协作：明确职责边界，减少冲突
- 可测试性：便于单元测试与集成测试
- 可复用性：组件化、模块化设计

MVP模式 (Model-View-Presenter)

- Model：数据层，负责数据获取与存储
- View：视图层，负责UI展示与用户交互
- Presenter：主持人，协调Model与View
- 特点：View与Model完全解耦，通过Presenter通信
- 优点：职责清晰、易于测试
- 缺点：Presenter容易过于臃肿，接口较多
- 适用场景：中等复杂度的业务应用

MVVM模式 (Model-View-ViewModel)

- Model：数据层，数据模型与业务逻辑
- View：视图层，UI展示 (Activity/Fragment/Widget)
- ViewModel：视图模型，管理视图状态与业务逻辑
- 核心机制：数据绑定 (Data Binding)
- 特点：View与ViewModel双向绑定，自动同步
- 优点：代码量少、声明式UI、响应式编程
- 缺点：调试难度稍大，学习曲线较陡
- 适用场景：中大型项目、交互复杂的应用

MVP vs MVVM 对比

维度	MVP	MVVM
通信方式	接口调用	数据绑定
View与Presenter/ViewModel关系	一对一	一对多
代码量	较多	较少
测试难度	中等	较低
学习成本	较低	较高
适用规模	中小型	中大型

分层架构设计原则

- 表现层 (Presentation Layer) : UI组件、ViewModel/Presenter
- 业务层 (Domain Layer) : 用例、业务规则、领域模型
- 数据层 (Data Layer) : 仓库、数据源、数据模型
- 依赖原则: 上层依赖下层, 下层不依赖上层
- 单一职责: 每层只负责一件事
- 开闭原则: 对扩展开放, 对修改关闭

架构选型决策树

- 简单项目 → MVC或直接架构
- 中等项目 → MVP
- 大型项目 → MVVM + 分层架构
- 团队因素 → 团队技术栈与熟悉程度

3. 数据存储方案 (15分钟)

移动应用数据存储全景图

- 轻量配置: 键值对存储
- 结构化数据: 关系型数据库
- 文件存储: 文件系统
- 云端存储: 后端服务

SharedPreferences / UserDefaults

- 定位: 轻量级键值对存储
- 适用场景: 用户偏好设置、配置项、缓存标记
- Android: SharedPreferences / DataStore
- iOS: UserDefaults

- 小程序: wx.setStorageSync
- 鸿蒙: Preferences
- 优点: 使用简单、性能好
- 缺点: 不支持复杂数据、存储量有限

SQLite 关系型数据库

- 定位: 嵌入式关系型数据库
- 适用场景: 大量结构化数据、复杂查询、离线数据
- 特点: 轻量级、零配置、事务支持、SQL标准
- Android: 原生SQLite API
- iOS: Core Data / FMDB
- 跨平台: sqflite (Flutter)

Room 持久化库 (Android)

- 定位: SQLite的ORM封装
- 核心组件: Entity、DAO、Database
- 优点: 编译时检查、简化代码、与LiveData集成
- 代码示例: Entity定义、DAO接口、Database类

Core Data (iOS)

- 定位: iOS官方对象图管理框架
- 核心概念: NSManagedObject、NSFetchRequest、NSPersistentContainer
- 特点: 与UIKit深度集成、内存管理优化
- 适用场景: iOS原生应用复杂数据管理

小程序本地缓存

- 同步API: wx.setStorageSync / wx.getStorageSync
- 异步API: wx.setStorage / wx.getStorage
- 存储限制: 单个key最大1MB, 总上限10MB
- 适用场景: 用户会话、页面状态缓存

鸿蒙数据管理

- Preferences: 轻量级键值存储
- 关系型数据库: 关系数据存储
- 分布式数据: 跨设备数据同步
- 特点: 统一数据管理框架、分布式能力

存储方案选型指南

方案	适用场景	数据量	性能	复杂度
SharedPreferences	配置项、小数据	小	高	低
SQLite/Room	结构化数据、复杂查询	中	中	中
文件存储	图片、文档、媒体	大	中	低
云端存储	跨设备、共享数据	大	网络依赖	高

4. 第一学时小结（5分钟）

- 总结MVP与MVVM架构模式的核心思想与适用场景
- 回顾分层架构设计原则与各层职责
- 梳理数据存储方案的选型策略
- 答疑与过渡：引出下节课的平台工程实践内容

第二学时：移动平台工程实践（45分钟）

5. 权限管理与运行时权限请求（12分钟）

权限管理概述

- 为什么需要权限？保护用户隐私与系统安全
- 权限分类：普通权限、危险权限、特殊权限
- 发展趋势：从安装时授权到运行时授权

Android 权限管理

- 正常权限：在AndroidManifest中声明即可
- 危险权限：需要运行时动态申请
- 日历、相机、联系人、位置、麦克风、电话、传感器、短信、存储
- 权限申请流程：
 1. 检查权限是否已授予
 2. 未授予则请求权限
 3. 处理用户授权结果
 4. 被拒绝时的降级处理
- 最佳实践：
 - 按需申请，不要一次性申请所有权限
 - 申请前解释用途（Rationale）
 - 被永久拒绝后引导用户到设置页面

iOS 权限管理

- 权限类型：相机、相册、位置、麦克风、通知等
- Info.plist 配置：必须声明用途描述（Purpose String）
- 权限申请：调用对应API触发系统弹窗
- 特点：只能申请一次，拒绝后需引导用户到设置

小程序权限管理

- 接口权限：部分API需要用户授权
- 授权方式：wx.authorize
- 授权范围：scope.userInfo、scope.location、scope.camera等
- 注意：用户拒绝后需引导重新授权

权限管理最佳实践

- 最小权限原则：只申请必要的权限
- 透明原则：清晰告知权限用途
- 优雅降级：权限被拒后提供替代方案
- 用户体验：避免频繁打扰，在需要时才申请

6. 应用生命周期管理（10分钟）

为什么要关注生命周期？

- 避免内存泄漏
- 正确保存与恢复状态
- 合理管理资源
- 提升用户体验

Android 应用生命周期

- Activity生命周期：
onCreate → onStart → onResume → onPause → onStop → onDestroy
- 关键回调的触发时机与职责
- Fragment生命周期：与Activity类似但有额外状态
- Application生命周期：全局状态管理
- 常见问题：
 - 屏幕旋转导致的数据丢失
 - 后台资源未释放
- 生命周期感知组件（Lifecycle）

iOS 应用生命周期

- AppDelegate：应用级生命周期

- 应用状态: Not Running → Inactive → Active → Background → Suspended
- ViewController生命周期:
- viewDidLoad → viewWillAppear → viewDidAppear → viewWillDisappear → viewDidDisappear
- 场景 (Scene) 生命周期: iPad多窗口支持

小程序生命周期

- 应用生命周期: onLaunch → onShow → onHide
- 页面生命周期: onLoad → onShow → onReady → onHide → onUnload
- 页面路由与页面栈管理

生命周期管理最佳实践

- 在正确的时机注册与注销监听器
- 重要数据及时持久化
- 避免在生命周期回调中做耗时操作
- 使用生命周期感知组件 (如AndroidX Lifecycle)

7. 后台任务处理 (10分钟)

后台任务概述

- 什么是后台任务? 应用不在前台时仍需执行的任务
- 后台限制: 系统为了省电与性能, 对后台任务有限制
- 常见场景: 数据同步、推送处理、定时任务、音乐播放

Android 后台任务方案

- Service: 传统后台服务 (注意: Android 8.0后台服务限制)
- WorkManager: 推荐的后台任务调度框架
- 特点: 兼容不同版本、支持约束条件、持久化任务
- 适用场景: 可延迟的后台任务
- JobScheduler: 系统级任务调度
- Foreground Service: 前台服务 (必须显示通知)
- 后台限制:
- Android 6.0: 低电耗模式 (Doze)
- Android 8.0: 后台执行限制
- Android 12: 更严格的后台启动限制

iOS 后台任务

- 后台模式: 音频、定位、VoIP、后台获取、远程通知
- Background Tasks框架: BGAppRefreshTask、BGProcessingTask

- 特点：时间受限、系统调度
- 最佳实践：尽量减少后台执行时间

小程序后台运行

- 后台音乐播放：backgroundAudioManager
- 后台定位：持续定位
- 限制：大部分功能在后台暂停

后台任务最佳实践

- 优先选择官方推荐的方案
- 合理设置任务约束（充电、WiFi、空闲时）
- 及时释放资源，避免耗电
- 做好异常处理与重试机制

8. 推送通知与近场通信（8分钟）

推送通知集成

- 作用：提高用户活跃度、传递重要信息
- Android推送：
 - FCM（Firebase Cloud Messaging）
 - 国内厂商推送：华为、小米、OPPO、vivo推送
 - 个推、极光等第三方推送
- iOS推送：APNs（Apple Push Notification service）
- 小程序：订阅消息、模板消息
- 推送最佳实践：
 - 精准推送，避免骚扰
 - 丰富的通知样式
 - 点击跳转与深链接
 - 用户可关闭推送

蓝牙通信基础

- 蓝牙经典 vs BLE（低功耗蓝牙）
- BLE核心概念：
 - 中心设备（Central） vs 外设（Peripheral）
 - Service、Characteristic、Descriptor
 - GATT协议
- 典型场景：智能手环、智能家居、Beacon
- 开发步骤：
 1. 检查蓝牙权限与状态

2. 扫描外设
3. 连接设备
4. 发现服务与特征
5. 读写数据、监听通知

WiFi通信基础

- WiFi直连 (WiFi P2P) : 设备间直接通信
- 本地网络服务发现: mDNS/Bonjour
- 热点通信: 通过同一局域网通信
- 适用场景: 文件传输、设备控制、多人游戏

9. 课程思政融入 (5分钟)

- **案例:** 国产移动操作系统的数据安全与隐私保护设计
- **思考:** 为什么数据安全是移动应用的生命线?
- **讨论:** 作为开发者, 如何在架构设计中体现对用户隐私的尊重?

五、学前测验

测验说明

本测验旨在检测学习者对移动应用架构与平台工程实践的前期认知程度, 帮助确定学习起点。

测验题目

第1题: 在MVVM架构模式中, View与ViewModel之间的通信主要依靠什么机制?

- A. 接口回调
- B. 数据绑定
- C. 事件总线
- D. 广播通知

正确答案: B

答案解析: MVVM的核心特征是数据绑定 (Data Binding), View与ViewModel之间通过绑定关系自动同步数据状态, 无需手动调用接口。这是MVVM与MVP的主要区别之一。

第2题: 以下哪种数据存储方案最适合存储用户偏好设置 (如主题、语言等配置项)?

- A. SQLite
- B. 文件存储

C. SharedPreferences

D. 网络存储

正确答案：C

答案解析：SharedPreferences (Android) / UserDefaults (iOS) 是专门用于轻量级键值对存储的方案，适合存储配置项、用户偏好等小数据，使用简单且性能好。

第3题：关于Android运行时权限，以下说法错误的是？

A. 危险权限需要在运行时动态申请

B. 所有权限都只能申请一次

C. 申请权限前应向用户解释用途

D. 权限被拒后应提供降级方案

正确答案：B

答案解析：Android运行时权限可以多次申请，用户可以选择"不再询问"。如果用户选择了"不再询问"，则无法再通过系统弹窗申请，需要引导用户到设置页面手动开启。

第4题：在Android中，以下哪个组件是官方推荐的可延迟后台任务调度方案？

A. Service

B. BroadcastReceiver

C. WorkManager

D. AsyncTask

正确答案：C

答案解析：WorkManager是Android Jetpack中的后台任务调度组件，兼容不同Android版本，支持约束条件（如充电、WiFi），任务持久化，是官方推荐的后台任务方案。

第5题：以下关于BLE（低功耗蓝牙）的描述，哪个是正确的？

A. BLE适合传输大文件

B. BLE功耗比经典蓝牙更高

C. BLE采用GATT协议进行数据通信

D. BLE只支持Android设备

正确答案：C

答案解析：BLE采用GATT（Generic Attribute Profile）协议进行通信，通过Service、Characteristic、Descriptor的层级结构组织数据。BLE适合低功耗、小数据量的场景，功耗比经典蓝牙低，Android和iOS都支持。

测验结果评估

- **5题全对：**优秀，您对移动应用架构与工程实践有很好的基础

- **3-4题正确**：良好，具备基本概念，需要深化理解
- **1-2题正确**：及格，需要系统学习架构与工程实践知识
- **0题正确**：需努力，建议从基础概念开始学习

六、课后作业与课程总结

课程总结

1. **架构设计**是项目可维护性的基石，MVP/MVVM是移动应用主流架构模式
2. **分层架构**遵循单一职责与依赖倒置原则，使代码更清晰
3. **数据存储**需根据数据类型与场景选择合适方案
4. **权限管理**应遵循最小权限原则，注重用户体验
5. **生命周期管理**直接影响应用稳定性与用户体验
6. **后台任务**受系统限制，需选择合适的方案并优化

课后作业

1. 选择一个你熟悉的移动应用（如外卖、购物、社交类），分析它可能采用的架构模式，并画出分层架构图
2. 对比SQLite、SharedPreferences、文件存储三种方案，用表格列出它们的优缺点和适用场景
3. 思考：如果要开发一个运动记录APP，需要申请哪些权限？在什么时机申请比较合适？
4. 调研一款智能手环或智能家居设备，描述其蓝牙通信的可能流程

下节预告

- 性能优化：启动速度、内存、网络、UI渲染全方位优化
- 应用发布流程：从开发到上架的完整链路
- 代码质量保障：规范、静态分析、自动化测试
- Git团队协作：分支管理与代码审查
- AI工具深度应用：重构、优化、测试生成

实验预告

- 实验六：跨平台综合项目实战

- 内容：基于架构模式重构项目，集成数据存储与权限管理
- 要求：完成MVP/MVVM架构改造，实现数据持久化

七、教学反思

(课后填写)